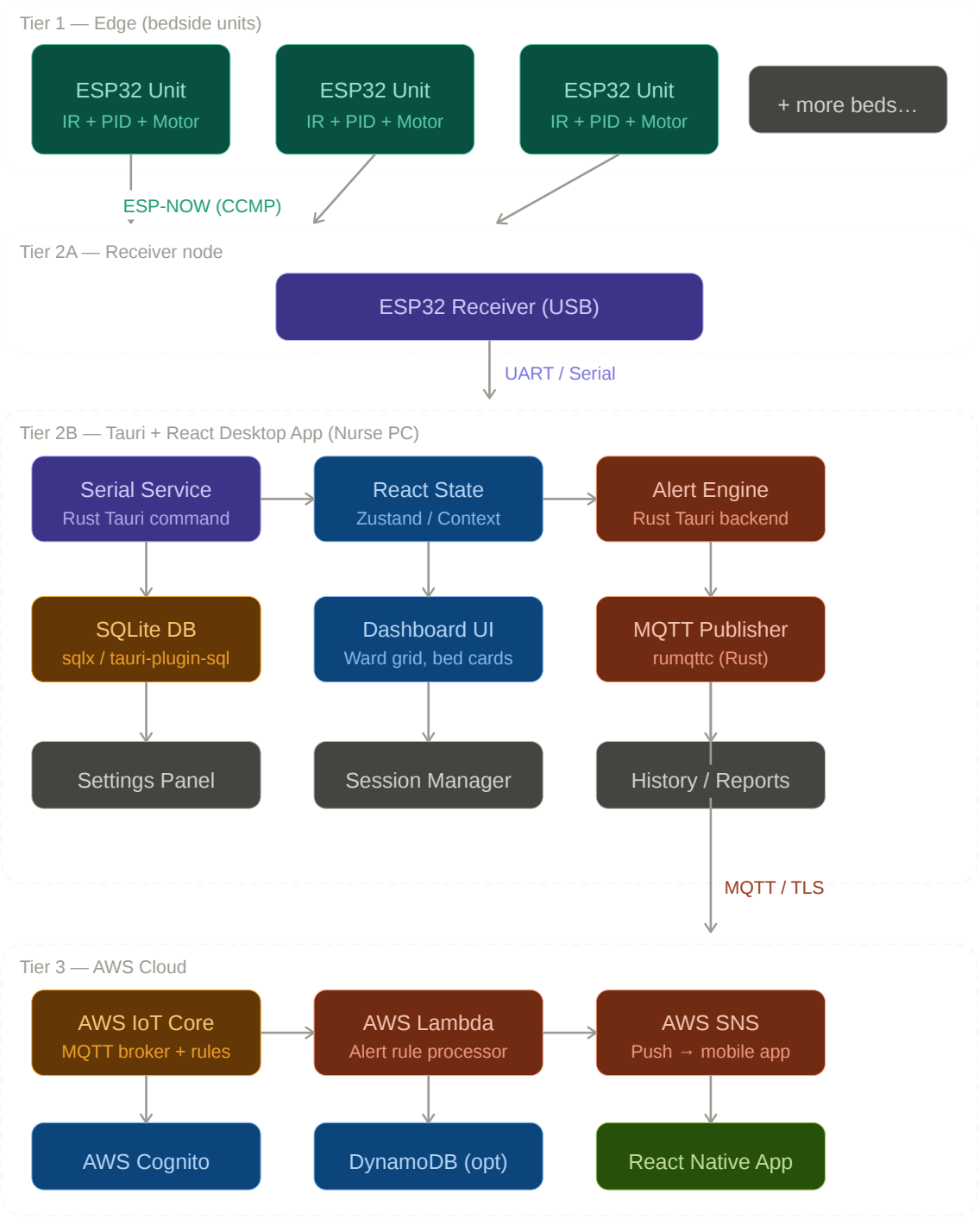
Desktop app: Tauri + React

System design for our Smart IV desktop app — Covering the architecture, data flow, DB schema, and cloud integration layer.



Tauri app structure

```
smart-iv-desktop/
├── src-tauri/
│   ├── src/
│   │   ├── main.rs
│   │   ├── serial.rs        # Serial port listener (serialport crate)
│   │   ├── db.rs            # SQLite via sqlx
│   │   ├── mqtt.rs          # rumqttc MQTT publisher
│   │   ├── alert.rs         # Alert rule engine
│   │   └── commands.rs      # Tauri IPC commands exposed to React
│   └── Cargo.toml
├── src/
│   ├── store/               # Zustand slices
│   │   ├── bedsStore.ts     # Live bed state
│   │   └── alertStore.ts
│   ├── components/
│   │   ├── BedCard.tsx
│   │   ├── WardGrid.tsx
│   │   └── AlertBanner.tsx
│   ├── pages/
│   │   ├── Dashboard.tsx
│   │   ├── History.tsx
│   │   └── Settings.tsx
│   └── lib/
│       └── tauriEvents.ts   # listen() wrappers for Tauri events
└── package.json
```

SQLite schema

These are the four tables we need to get started:

```cpp
-- Static config per physical device
CREATE TABLE beds (
  bed_id       TEXT PRIMARY KEY,   -- e.g. "03"
  patient_name TEXT,
  ward         TEXT,
  drop_factor  INTEGER DEFAULT 20,
  mac_address  TEXT,
  created_at   DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- One row per infusion session
CREATE TABLE sessions (
```

```sql
  session_id    TEXT PRIMARY KEY,   -- UUID generated on de
sktop
  bed_id        TEXT REFERENCES beds(bed_id),
  max_volume_ml REAL NOT NULL,
  target_ml_hr  REAL NOT NULL,
  started_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
  ended_at      DATETIME,
  end_reason    TEXT    -- 'COMPLETED' | 'CANCELLED' | 'ERR
OR'
);

-- Telemetry ring buffer — the live feed from ESP32s
CREATE TABLE telemetry (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  bed_id         TEXT REFERENCES beds(bed_id),
  session_id     TEXT REFERENCES sessions(session_id),
  ts             DATETIME DEFAULT CURRENT_TIMESTAMP,
  flow_rate_ml   REAL,
  vol_remaining  REAL,
  battery_pct    INTEGER,
  status         TEXT    -- 'STABLE' | 'BLOCKAGE' | 'EMPTY'
| 'CONN_LOST'
);

-- Audit log for every alert event
CREATE TABLE alerts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  bed_id      TEXT REFERENCES beds(bed_id),
  session_id  TEXT,
  ts          DATETIME DEFAULT CURRENT_TIMESTAMP,
  alert_type  TEXT,   -- 'BLOCKAGE' | 'EMPTY_BAG' | 'CONN_L
OST' | 'BATTERY_LOW'
  resolved_at DATETIME,
  resolved_by TEXT    -- nurse login name
);

-- Indexes for dashboard queries
CREATE INDEX idx_telemetry_bed_ts ON telemetry(bed_id, ts D
```

```
ESC);
CREATE INDEX idx_alerts_unresolved ON alerts(resolved_at) W
HERE resolved_at IS NULL;
```

> Keep telemetry to the last 7 days with a purge job on app startup — SQLite doesn't need to be a data warehouse.

Data flow (end-to-end)

Step 1 — Serial ingestion (Rust)

A Tauri background thread opens the COM/tty port and reads JSON lines in a loop. On each valid packet it: writes a row to `telemetry`, runs the alert rules, emits a Tauri event to the React frontend, and — if an alert fires — queues a message for the MQTT publisher.

```rust
// src-tauri/src/serial.rs (sketch)
let packet: BedPacket = serde_json::from_str(&line)?;
db.insert_telemetry(&packet).await?;
alert_engine.evaluate(&packet, &db).await?;
app_handle.emit_all("bed-update", &packet)?;
```

Step 2 — React state update

The frontend listens for Tauri events and pushes them into Zustand:

```typescript
// src/lib/tauriEvents.ts
import { listen } from '@tauri-apps/api/event';
import { useBedsStore } from '../store/bedsStore';

listen<BedPacket>('bed-update', ({ payload }) => {
  useBedsStore.getState().upsertBed(payload);
});
```

Step 3 — MQTT publish (Rust)

The MQTT publisher runs as a separate Tauri managed state using `rumqttc`. On alert events, it publishes to a structured topic:

```
smartiv/{ward}/{bed_id}/telemetry   ← continuous telemetry
smartiv/{ward}/{bed_id}/alert       ← alert events only
```

Both with QoS 1 and TLS. AWS IoT Core receives these.

Step 4 — AWS IoT Core → Lambda → SNS

Set up an IoT Rule that triggers when `status != 'STABLE'`:

```sql
-- IoT Rule SQL
SELECT *, topic(2) as ward, topic(3) as bedId
FROM 'smartiv/+/+/alert'
WHERE status <> 'STABLE'
```

This fires a Lambda that formats and sends an SNS push notification to your React Native app subscribers.

AWS setup checklist

```
AWS IoT Core
  ├── Thing per nurse station (one per hospital ward)
  ├── Certificate + Policy attached (allow iot:Connect, iot:Publish)
  ├── IoT Rule: SQL filter on /alert topic → Lambda target
  └── IoT Rule: SQL filter on /telemetry → DynamoDB (optional, for cloud history)

IAM
  └── Lambda execution role: SNS:Publish permission

SNS
  └── Topic: smartiv-alerts
      └── Subscriptions: mobile app (via AWS Amplify push) + optional SMS

Cognito
  └── User pool for nurse/doctor login in mobile app
  └── Identity pool for mobile app ↔ IoT Core subscription
```

Key Rust crates to add

```toml
# Cargo.toml
[dependencies]
tauri           = { version = "2", features = ["shell-open"]
```

```
}
sqlx            = { version = "0.7", features = ["sqlite",
"runtime-tokio"] }
serialport      = "4"
rumqttc         = "0.24"          # MQTT client with TLS
serde           = { version = "1", features = ["derive"] }
serde_json      = "1"
tokio           = { version = "1", features = ["full"] }
uuid            = { version = "1", features = ["v4"] }
```

One gotcha to plan for

The serial reader and MQTT publisher both need to survive independently — if AWS goes offline, the serial reader and dashboard must keep working. Structure them as separate `tokio::spawn` tasks that don't share a single error boundary. If MQTT reconnect fails, log it and buffer the last N alert payloads in memory to retry, but never block the serial ingestion path.

This gives us full local-first resilience as designed in the Smart IV spec.